K-means Analysis

Sunghyun Ahn Megan Dhillon Yoohan Park
Breitling Snyder

Thursday 18th December, 2025

Curvenote

```
import pandas as pd
import geopandas as gpd
import matplotlib.pyplot as plt

import sys
sys.path.append("..")
from CEStools.utils import clean_data

import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Import and clean the data
df_raw = pd.read_csv('../data/calenviroscreen40.csv')

df_clean = clean_data(df_raw)
X = df_clean[df_clean.notna().all(axis=1)]
```

- CalEnviroScreen uses `-999` and `-1998` as *sentinel values* meaning "missing" or "not applicable".
- We create a boolean mask that keeps only rows where **all** entries do not include `-999` and `-1998`.
- The cleaned DataFrame is stored in `X`.
- We print the number of original vs. cleaned rows to see how many tracts were removed.

```
print(f"Original rows: {len(df_clean):,}")
print(f"Clean rows   : {len(X):,}")
```

```
Original rows: 8,035
Clean rows   : 7,354
```

```
X
```

	[illegible]
0	[illegible]
1	[illegible]
2	[illegible]
3	[illegible]
4	[illegible]
...	...
8030[illegible]	[illegible]
8031[illegible]	[illegible]
8032[illegible]	[illegible]
8033[illegible]	[illegible]
8034[illegible]	[illegible]

7354 rows × 30 columns

- K-means is distance-based, so features with larger scales can dominate the distance computation.
- We use `StandardScaler` to transform each feature to have:
  - mean ≈ 0
  - standard deviation ≈ 1
- The scaled feature matrix is stored in `X_scaled`.
- For each `k` from 2 to 10:
  - Fit a `KMeans` model on `X_scaled` with:
    - `random_state=42` for reproducibility
    - `n_init=10` to run the algorithm multiple times and keep the best solution.
  - Store the **inertia** (within-cluster sum of squared distances) in `inertias`.
  - Compute the **silhouette score** (how well-separated the clusters are) and store it in `sil_scores`.

```
# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X.values)
```

```
# Run KMeans for k = 2,...,10 and compute inertia + silhouette score
k_values = list(range(2, 11))
inertias = []
sil_scores = []

for k in k_values:
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,  # set random state for reproducibility
        n_init=10
    )
    labels = kmeans.fit_predict(X_scaled)
    inertias.append(kmeans.inertia_)
    sil_scores.append(silhouette_score(X_scaled, labels))

# Plot: Elbow (Inertia) + Silhouette on twin axes
fig, ax1 = plt.subplots()

# Inertia line (left y -axis)
ax1.plot(
    k_values, inertias,
    marker="o",
    color="tab:blue",
    label="Inertia"
)
ax1.set_xlabel("Number of Clusters k")
ax1.set_ylabel("Inertia", color="tab:blue")
ax1.tick_params(axis="y", labelcolor="tab:blue")
ax1.set_title("Elbow Method vs. Silhouette Score")
ax1.grid(alpha=0.3, linestyle=" - -")  # light grid

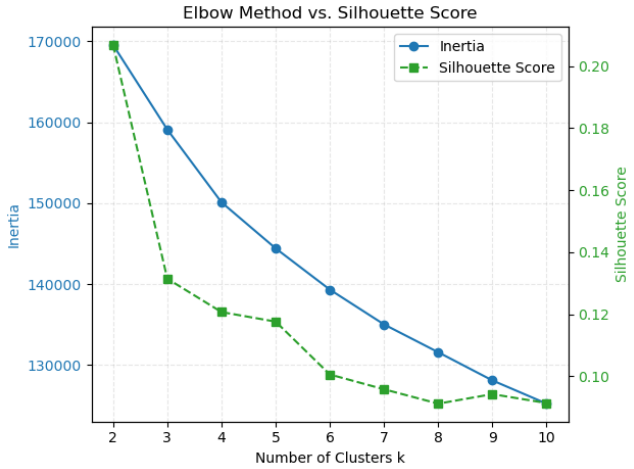
# Silhouette line (right y -axis)
ax2 = ax1.twinx()
ax2.plot(
    k_values, sil_scores,
    marker="s",
    linestyle=" - -",
    color="tab:green",
    label="Silhouette Score"
)
ax2.set_ylabel("Silhouette Score", color="tab:green")
ax2.tick_params(axis="y", labelcolor="tab:green")

# Combine legends from both axes
lines1, labels1 = ax1.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()
```

```
ax1.legend(lines1 + lines2, labels1 + labels2, loc="best")

plt.tight_layout()
plt.savefig("../visualizations/kmeans_visualizations/elbowvssilohuette_chart.png", dpi=300,

plt.show()
```



1 Interpretation: Elbow & Silhouette Plot

- The **inertia curve** (blue) decreases as we increase `k`, with diminishing returns after a certain point.
- The **silhouette curve** (green) shows how well-separated the clusters are for each `k`.
- We choose the value of `k` with the highest silhouette score (here, `k = 2`), balancing model fit and cluster separation.

```
feature_cols = ['CIscoreP', 'OzoneP',  'PM2_5_P', 'DieselPM_P', 'PesticideP',
                'Tox_Rel_P', 'TrafficP', 'DrinkWatP', 'Lead_P', 'CleanupP',
                'GWThreatP', 'HazWasteP', 'ImpWatBodP', 'SolWasteP', 'PolBurdP',
                'AsthmaP', 'LowBirWP', 'CardiovasP', 'EducatP', 'Ling_IsolP',
                'PovertyP', 'UnemplP', 'HousBurdP', 'PopCharP',
               'Hispanic', 'White', 'AfricanAm', 'NativeAm', 'OtherMult', 'AAPI']

# Fit final model with best k (=2) based on silhouette
best_k = k_values[sil_scores.index(max(sil_scores))]
print(f"\nBest k based on silhouette score: {best_k}")

final_kmeans_2 = KMeans(
```

```
    n_clusters=best_k,
    random_state=42,
    n_init=10
)
final_labels_2 = final_kmeans_2.fit_predict(X_scaled)

cluster_centers = pd.DataFrame(
    final_kmeans_2.cluster_centers_,
    columns=feature_cols
)

cluster_centers
```

```
Best k based on silhouette score: 2
```

	[illegible]
0	[illegible] -0.033506 [illegible]
1	[illegible] ... [illegible]

2 rows × 30 columns

2 Interpretation : Cluster centers for k=2 (standardized scale)

- **Cluster 0**: shows positive standardized scores across CalEnviroScreen percentiles for pollution burden (e.g., PM2.5, Diesel PM, Pollution Burden), health outcomes (AsthmaP, CardiovasP), and socioeconomic vulnerability (PovertyP, UnemplP, PopCharP), while having a strongly negative deviation for White population share. This cluster can be interpreted as high-burden, high-vulnerability, minority-majority communities.
- **Cluster 1**: in contrast, exhibits negative standardized values for most environmental, health, and vulnerability indicators, and a positive deviation for White population share. This cluster corresponds to low-burden, more advantaged, White-majority communities.
- We select the value of `k` = 2 that gives the **maximum silhouette score**.
- Using this `best_k`, we fit a final `KMeans` model on `X_scaled`:
  - The final cluster assignments for each tract are stored in `final_labels`.
  - The standardized cluster means (centers) are stored in `cluster_centers` as a DataFrame.

- Each entry in `cluster_centers` is a **z-score**:
  - Positive values (>0) mean the cluster has above-average levels for that feature.
  - Negative values (<0) mean below-average levels for that feature.

```
# 1) Convert cluster centers into a DataFrame
cluster_centers = pd.DataFrame(
    final_kmeans_2.cluster_centers_,
    columns=feature_cols
)

# 2) Sort features by how different they are across clusters
#    (sort by variance across cluster means, descending)
feature_order = cluster_centers.var(axis=0).sort_values(ascending=False).index
cluster_centers_ord = cluster_centers[feature_order]

# 3) Draw the heatmap

fig, ax = plt.subplots(figsize=(min(0.5 * len(feature_order) + 3, 18), 3 + best_k))

im = ax.imshow(cluster_centers_ord, aspect="auto")

# Set x and y tick labels

ax.set_xticks(range(len(feature_order)))
ax.set_xticklabels(feature_order, rotation=90)

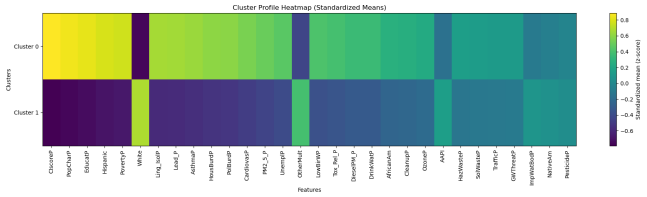
ax.set_yticks(range(best_k))
ax.set_yticklabels([f"Cluster {i}" for i in range(best_k)])

ax.set_title("Cluster Profile Heatmap (Standardized Means)")
ax.set_xlabel("Features")
ax.set_ylabel("Clusters")

# Add colorbar
cbar = fig.colorbar(im, ax=ax)
cbar.set_label("Standardized mean (z -score)")

plt.tight_layout()
plt.savefig("../visualizations/kmeans_visualizations/cluster_heatmap_2.png", dpi=300, bbox_i

plt.show()
```



2.1 Cluster profile heatmap (k=2)

The heatmap above shows the standardized cluster means (z-scores) for all CalEnviroScreen indicators and race-composition variables.

- Each row corresponds to one cluster (Cluster 0 and Cluster 1).
- Each column is a feature (e.g., CIscoreP, PolBurdP, PovertyP, PM2_5_P, White, etc.).
- Colors represent standardized means:
  - Yellow / light colors indicate **above-average** values (positive z-scores).
  - Purple / dark colors indicate **below-average** values (negative z-scores).

We see a very clear pattern:

- **Cluster 0** has high values for `CIscoreP`, `PolBurdP`, `PM2_5_P`, `DieselPM_P`, `AsthmaP`, `CardiovasP`, `PovertyP`, `UnemplP`, `PopCharP`, and other vulnerability metrics, while having a strongly negative value for White.
  → *This cluster represents* **high-burden, high-vulnerability, minority-majority communities**. **Cluster 1** *most environmental and socioeconomic burden indicators are below average, while* `White` *is above average.*
  → *This cluster represents* **low-burden, more advantaged, White-majority communities**.
  Features near zero in both clusters (e.g., GWThreatP, ImpWatBodP, PesticideP) do not strongly differentiate between clusters.

- ```
  final_kmeans_4 = KMeans(
      n_clusters=4,
      random_state=42,
      n_init=10
  )
  final_labels_4 = final_kmeans_4.fit_predict(X_scaled)

  cluster_centers = pd.DataFrame(
      final_kmeans_4.cluster_centers_,
      columns=feature_cols
  )

  cluster_centers
  ```
  ```

	CIscore	[illegible]	...	[illegible]
0	[illegible]	[illegible] 0.451294 [illegible] - - -	...	[illegible] 0.087735 [illegible] 0.377593
1	[illegible] - - - -	0.089290 [illegible] - - -	...	- - - - - [illegible] 1.384304 [illegible] 0.108950
2	[illegible]	[illegible] 0.195983 [illegible]	[illegible]	[illegible] 0.287438 [illegible]
3	- - [illegible] 0.126[illegible]	[illegible] 0.279100 [illegible] 0.290 [illegible]	[illegible]	- - - - [illegible]

4 rows × 30 columns

```
# 1) Convert cluster centers into a DataFrame
cluster_centers = pd.DataFrame(
    final_kmeans_4.cluster_centers_,
    columns=feature_cols
)

# 2) Sort features by how different they are across clusters
#    (sort by variance across cluster means, descending)
feature_order = cluster_centers.var(axis=0).sort_values(ascending=False).index
cluster_centers_ord = cluster_centers[feature_order]

# 3) Draw the heatmap

fig, ax = plt.subplots(figsize=(min(0.5 * len(feature_order) + 3, 18), 3 + 4))

im = ax.imshow(cluster_centers_ord, aspect="auto")

# Set x and y tick labels

ax.set_xticks(range(len(feature_order)))
ax.set_xticklabels(feature_order, rotation=90)

ax.set_yticks(range(4))
ax.set_yticklabels([f"Cluster {i}" for i in range(4)])

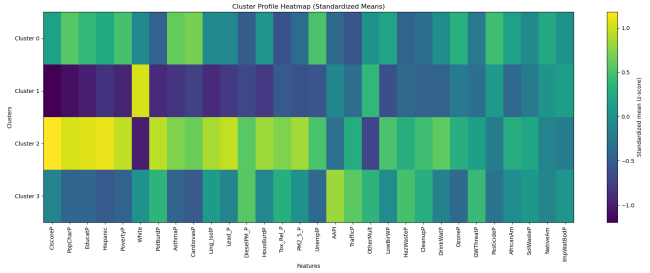
ax.set_title("Cluster Profile Heatmap (Standardized Means)")
ax.set_xlabel("Features")
ax.set_ylabel("Clusters")

# Add colorbar
cbar = fig.colorbar(im, ax=ax)
cbar.set_label("Standardized mean (z -score)")

plt.tight_layout()
```

```
plt.savefig("../visualizations/kmeans_visualizations/cluster_heatmap_4.png", dpi=300, bbox_i

plt.show()
```



2.2 Cluster profile heatmap (k=4)

The heatmap above shows the standardized cluster means (z-scores) for all CalEnviroScreen indicators and race-composition variables.

- Each row corresponds to one cluster (Clusters 0, 1, 2, and 3).
- Each column is a feature (e.g., CIscoreP, PolBurdP, PovertyP, PM2_5_P, White, etc.).
- Colors represent standardized means:
  - Yellow / light colors indicate **above-average** values (positive z-scores).
  - Purple / dark colors indicate **below-average** values (negative z-scores).

```
shp_path = "../data/calenviroscreen40_shp/CES4 Final Shapefile.shp"
shape = gpd.read_file(shp_path)

shape = clean_data(shape, keep_geom=True)
shape = shape[df_clean.notna().all(axis=1)]
shape = shape.to_crs(epsg=3857)

cluster_labels_2 = pd.Series(final_labels_2, index=X.index, name="cluster_2")
cluster_labels_4 = pd.Series(final_labels_4, index=X.index, name="cluster_4")

clusters_df = pd.concat(
    [cluster_labels_2, cluster_labels_4],
    axis=1
)

gdf = shape.join(clusters_df)
```

```
ax = gdf.plot(
    column="cluster_2",
    categorical=True,
    legend=True,
    figsize=(10, 12),
    linewidth=0.05,
    edgecolor="white"
)

ax.set_title("California Census Tracts by K -Means Cluster")
ax.set_axis_off()

plt.savefig("../visualizations/kmeans_visualizations/cluster_map_2.png", dpi=300, bbox_inche
plt.show()
```


```
ax = gdf.plot(
    column="cluster_4",
    categorical=True,
    legend=True,
    figsize=(10, 12),
    linewidth=0.05,
    edgecolor="white"
```

```
)

ax.set_title("California Census Tracts by K -Means Cluster")
ax.set_axis_off()

plt.savefig("../visualizations/kmeans_visualizations/cluster_map_4.png", dpi=300, bbox_inche
plt.show()
```